

Universidade do Minho
Escola de Engenharia

LiftPlan

Uma linguagem específica de domínio para programas de treino

Grupo 06

Vicente Silva, PG64042
Vítor Mina, PG64044

Engenharia Gramatical
Mestrado de Engenharia Informática

4 de novembro de 2026

Conteúdo

1 Conceção da Linguagem 1
1.1 Domínio e propósito 1
1.2 Conceitos e abstrações 1
1.3 Exemplos de programas válidos 2
2 Especificação Lexical e Sintática 6
2.1 Estrutura lexical 6
2.2 Gramática 6
2.3 Decisões de desenho 8
3 Especificação Semântica 9
3.1 Propriedades semânticas 9
3.2 Gramática de atributos 9
3.3 Ordem de cálculo 13
3.4 Volume garantido 13
3.5 Implementação 13
4 Qualidade da Linguagem e da Gramática 14
4.1 Qualidade da linguagem 14
4.2 Qualidade da gramática 15
4.3 Limitações 15

Índice de Tabelas

Tabela 1 Conceitos do domínio e construções da LiftPlan. 2
Tabela 2 Volume garantido do programa `UpperLower`. 4
Tabela 3 Elementos lexicais da LiftPlan. 6
Tabela 4 Ordem das operações, da menos prioritária para a mais prioritária. 8
Tabela 5 Propriedades semânticas que a sintaxe não decide. 9
Tabela 6 Atributos de cada símbolo. 10
Tabela 7 Avaliação da qualidade da linguagem. 14
Tabela 8 Avaliação da qualidade da gramática. 15

1 Conceção da Linguagem

1.1 Domínio e propósito

A **LiftPlan** é uma linguagem específica de domínio (*Domain-Specific Language*, DSL) para definir **programas de treino de força**. Um programa descreve o que um atleta deve fazer ao longo de várias semanas, organizado em rotinas, semanas, dias e sessões. Indica os exercícios, as séries, as repetições, a carga, a intensidade percebida (RPE) e os descansos.

Estes planos são hoje escritos em folhas de cálculo ou texto livre, que não detetam erros (um exercício nunca definido, um RPE fora da escala, uma semana em falta) nem calculam o volume de treino. A LiftPlan resolve isto: estrutura o plano, permite usar constantes e variáveis, adapta o treino com `last_rpe` e `fatigue`, verifica o plano antes de ser usado e calcula o seu volume.

O âmbito é só o treino de força, medido em séries, repetições e carga. Ficam de fora outros desportos e treinos medidos por tempo ou distância.

Destina-se a treinadores e atletas com conhecimentos básicos de programação, e a ferramentas que usam o plano (Parte II do projeto: representação, visualização e transformação dos programas).

1.2 Conceitos e abstrações

A Tabela 1 mostra os conceitos do domínio e a construção da linguagem que os representa.

Conceito	Construção	Descrição
Programa	`program ID { ... }`	Unidade de topo. Contém constantes, variáveis, exercícios e rotinas.
Exercício	`exercise ID { k: "v", ... }`	Entrada de catálogo com texto descritivo (categoria, equipamento, …). Tem pelo menos um atributo.
Rotina, semana, dia, sessão	`routine`, `week N`, `monday { ... }`, `session "nome"`	Estrutura do plano, como um calendário. As semanas são numeradas 1, 2, 3, … Cada dia aparece no máximo uma vez por semana e tem uma ou mais sessões, com nomes diferentes.
Prescrição	`do E sets S reps R load L rpe P;`	Executa o exercício `E`. Carga e RPE são opcionais; sem carga, assume-se o peso corporal.
Descanso	`rest T;`	Pausa com duração numérica.
Estado	`var`, `const`, atribuição	Valores com nome. As variáveis podem mudar; as constantes têm valor conhecido antes de executar.
Condição	`if (c) { ... } else { ... }`	Escolha entre dois caminhos, em geral com `last_rpe` ou `fatigue`.
Repetição	`repeat N times { ... }`	Repete um bloco `N` vezes.
Circuito	`circuit rounds N { ... }`	Repete um bloco de exercícios em `N` rondas. Funciona como `repeat`, mas indica a intenção (útil na Parte II).
Predefinidos	`last_rpe`, `fatigue`	Valores só de leitura: `last_rpe` (`float`) e `fatigue` (`int`).

Tabela 1: Conceitos do domínio e construções da LiftPlan.

A linguagem tem quatro tipos (`int`, `float`, `bool` e `string`). Um `int` pode ser usado onde se espera um `float`, mas não o contrário.

1.3 Exemplos de programas válidos

O primeiro exemplo descreve duas semanas de um plano "superior/inferior". Usa constantes (cargas máximas), `if`/`else` com `last_rpe`, `circuit`, `repeat` e cargas calculadas por expressões.

```
program UpperLower {
    const squat_1rm : float = 140.0;
    const bench_1rm : float = 100.0;

    exercise Squat            { category: "lower", equipment:
"barbell" }
    exercise RomanianDeadlift { category: "lower", equipment:
"barbell" }
    exercise BenchPress       { category: "upper", equipment:
"barbell" }
    exercise Row              { category: "upper", equipment:
"barbell" }
    exercise PullUp           { category: "upper", equipment:
"bodyweight" }

    routine Strength {
        week 1 {
            monday {
                session "Lower" {
                    var back_off : float = squat_1rm * 0.65;
                    do Squat sets 4 reps 5 load squat_1rm * 0.75 rpe
7;
                    rest 180;
                    if (last_rpe > 9.0) {
                        back_off = back_off - 5.0;
                    } else {
                        back_off = back_off + 2.5;
                    }
                    do Squat sets 2 reps 8 load back_off rpe 7;
                    rest 120;
                    do RomanianDeadlift sets 3 reps 8 load 90.0 rpe 7;
                }
            }
            thursday {
                session "Upper" {
                    do BenchPress sets 4 reps 6 load bench_1rm * 0.75
rpe 8;
                    rest 150;
                    circuit rounds 3 {
                        do Row sets 1 reps 10 load 60.0;
                        do PullUp sets 1 reps 6 load 10.0;
                        rest 90;
                    }
                }
            }
        }
        week 2 {
            monday {
                session "Lower" {
                    do Squat sets 4 reps 5 load squat_1rm * 0.75 + 5.0
rpe 8;
                    rest 180;
```

```
                    do RomanianDeadlift sets 3 reps 8 load 95.0 rpe 7;
                }
            }
            thursday {
                session "Upper" {
                    repeat 4 times {
                        do BenchPress sets 1 reps 6 load bench_1rm *
0.75 + 2.5 rpe 8;
                        rest 150;
                    }
                }
            }
        }
    }
}
```

Listagem 1: Programa `UpperLower` (ficheiro `valid.lp`).

O volume de uma prescrição é `sets * reps * load` (sem `load`, vale 1). Diz-se "garantido" porque só conta o que se sabe antes de executar. As cargas `squat_1rm * 0.75` e `bench_1rm * 0.75` são constantes e entram na conta (105.0 e 75.0). A série de `back_off` e o `if`/`else` não entram, porque dependem de `last_rpe`. Cada sessão pertence a um dia da semana, o que permitirá exportar o plano para um calendário na Parte II. Os volumes garantidos são:

Semana	**Sessão**	**Cálculo**	**Volume**
1	Lower (segunda)	`4*5*105.0 + 3*8*90.0`	4260.0
1	Upper (quinta)	`4*6*75.0 + 3*(1*10*60.0 + 1*6*10.0)`	3780.0
2	Lower (segunda)	`4*5*110.0 + 3*8*95.0`	4480.0
2	Upper (quinta)	`4*(1*6*77.5)`	1860.0
		Total do programa	14380.0

Tabela 2: Volume garantido do programa `UpperLower`.

O segundo exemplo é mais pequeno: tem duas sessões no mesmo dia e um exercício de peso corporal com carga extra.

```
program FullBody {
    exercise Squat  { category: "lower" }
    exercise PullUp { category: "upper", equipment: "bodyweight" }

    routine Basics {
        week 1 {
            monday {
                session "Morning" {
                    repeat 3 times {
                        do Squat sets 1 reps 8 load 60.0 rpe 7;
                        rest 120;
                    }
```

```
                }
                session "Evening" {
                    do PullUp sets 3 reps 6 load 5.0;
                    rest 90;
                }
            }
        }
    }
}
```

Listagem 2: Programa `FullBody`.

Os volumes garantidos são `3*(1*8*60.0) = 1440.0` na sessão da manhã e `3*6*5.0 = 90.0` na da tarde.

2 Especificação Lexical e Sintática

Este capítulo define as palavras e símbolos da LiftPlan (estrutura lexical) e as regras que dizem como se combinam num programa (gramática). A gramática está escrita em Lark e é analisada com LALR(1): o analisador lê o programa da esquerda para a direita, decide com uma só palavra de avanço e nunca volta atrás.

2.1 Estrutura lexical

A Tabela 3 mostra os tipos de elementos que a linguagem reconhece.

Token	**Padrão**
Palavra-chave	`program var const int float bool string exercise routine week session do sets reps load rpe rest if else repeat times circuit rounds and or not true false`
`ID`	`[A-Za-z_][A-Za-z0-9_]*`
`DAY`	`monday tuesday wednesday thursday friday saturday sunday`
`INT`	`[0-9]+`
`FLOAT`	`[0-9]+\.[0-9]+`
`STRING`	`"[^"\n]*"`
Operadores	`== != <= >= < > + - * / =`
Pontuação	`{ } ( ) ; : ,`
Comentários e espaços	`//` até ao fim da linha, espaços e quebras de linha (ignorados)

Tabela 3: Elementos lexicais da LiftPlan.

Decisões principais:

- As 28 palavras-chave são reservadas. `last_rpe` e `fatigue` não são: são nomes predefinidos.
- `80.0` é lido como um só número decimal. Não existem `.5` nem `5.`, e os números negativos usam o operador `-`.
- Os dias (`monday`, …) só contam como dias no início de um bloco de dia. Noutros sítios são nomes normais.
- Escreve-se `sets 4 reps 8` em vez de `4 x 8`, porque `x` podia confundir-se com um nome.

2.2 Gramática

A Listagem 3 define a estrutura do programa e das instruções, e a Listagem 4 as expressões. Em Lark, `?regra` não cria um nó quando tem um só filho, e `-> nome` dá um nome a uma alternativa.

```
start: "program" ID "{" decl* "}"

?decl: var_decl | const_decl | exercise_decl | routine_decl
```

```
var_decl: "var" ID ":" type "=" expr ";"
const_decl: "const" ID ":" type "=" expr ";"
?type: T_INT | T_FLOAT | T_BOOL | T_STRING

exercise_decl: "exercise" ID "{" attr ("," attr)* "}"
attr: ID ":" expr

routine_decl: "routine" ID "{" week+ "}"
week: "week" INT "{" day+ "}"
day: DAY "{" session+ "}"
session: "session" STRING block

block: "{" stmt* "}"

?stmt: var_decl | const_decl | assign | perform | rest
     | if_stmt | repeat | circuit

assign: ID "=" expr ";"
perform: "do" ID "sets" expr "reps" expr load? rpe? ";"
load: "load" expr
rpe: "rpe" expr
rest: "rest" expr ";"
if_stmt: "if" "(" expr ")" block ("else" block)?
repeat: "repeat" expr "times" block
circuit: "circuit" "rounds" expr block
```

Listagem 3: Estrutura do programa e instruções.

```
?expr: or_e
?or_e: and_e | or_e "or" and_e -> or_
?and_e: not_e | and_e "and" not_e -> and_
?not_e: rel_e | "not" not_e -> not_
?rel_e: add_e | add_e (EQ | NE | LE | GE | LT | GT) add_e -> rel
?add_e: mul_e
      | add_e PLUS mul_e  -> plus
      | add_e MINUS mul_e -> minus
?mul_e: unary_e
      | mul_e STAR unary_e  -> times
      | mul_e SLASH unary_e -> divide
?unary_e: atom | MINUS unary_e -> neg
?atom: INT     -> int_lit
     | FLOAT   -> float_lit
     | STRING  -> str_lit
     | "true"  -> true
     | "false" -> false
     | ID      -> var
     | "(" expr ")"
```

Listagem 4: Expressões, com a precedência definida por camadas.

Os terminais com nome (`EQ`, `PLUS`, `T_INT`, …) são definidos pelo seu símbolo, por exemplo `EQ: "=="`.

2.3 Decisões de desenho

A ordem das operações (Tabela 4) está definida na própria gramática, por camadas: quanto mais abaixo, mais cedo a operação é calculada. Assim, `1 + 2 * 3` dá `1 + (2 * 3)`. As comparações não podem ser encadeadas: `a < b < c` é um erro.

Nível	**Operadores**	**Agrupamento**	**Regra**
1	`or`	Da esquerda	`or_e`
2	`and`	Da esquerda	`and_e`
3	`not`	Prefixo	`not_e`
4	`== != <= >= < >`	Sem encadeamento	`rel_e`
5	`+ -`	Da esquerda	`add_e`
6	`* /`	Da esquerda	`mul_e`
7	`-` (unário)	Prefixo	`unary_e`

Tabela 4: Ordem das operações, da menos prioritária para a mais prioritária.

Outras decisões:

- **Chavetas obrigatórias.** Cada `else` pertence sempre ao `if` anterior, sem dúvidas.
- **Estrutura mínima.** Uma rotina tem pelo menos uma semana, uma semana pelo menos um dia, um dia pelo menos uma sessão e um exercício pelo menos um atributo.
- **O dia é um nível próprio** (`monday { ... }`). Espelha um calendário e permite várias sessões no mesmo dia.
- **A gramática só define a forma.** Declarações, tipos, ordem das semanas, dias e sessões repetidos e a escala do RPE são verificados à parte (Secção 3).

3 Especificação Semântica

A gramática do Secção 2 só verifica a **forma** de um programa. Não sabe, por exemplo, se um exercício foi declarado ou se um número é válido. Este capítulo lista essas verificações e define-as com uma gramática de atributos.

3.1 Propriedades semânticas

Grupo	**Propriedade**
Nomes	Um nome tem de ser declarado antes de ser usado.
	Um nome não pode ser declarado duas vezes no mesmo bloco. Num bloco interior pode repetir-se um nome de fora.
	Só variáveis podem receber um novo valor. `do` só aceita exercícios, e um exercício não é um valor.
Tipos	O valor inicial ou atribuído tem o tipo da variável (um `int` serve onde se espera um `float`).
	`sets` e `reps` são inteiros; `load`, `rpe` e `rest` são números.
	A condição de um `if` é `bool`; o número de repetições de `repeat` e `circuit` é inteiro.
	Contas só com números, `and`/`or`/`not` só com `bool`, atributos de exercício são texto.
	`<`, `<=`, `>`, `>=` exigem números; `==` e `!=` aceitam também dois valores do mesmo tipo.
Valores	Não se divide por uma constante igual a zero. Um `rpe` conhecido está entre 1 e 10.
Estrutura	As semanas são numeradas 1, 2, 3, … por ordem.
	Cada dia aparece no máximo uma vez por semana; os nomes das sessões de um dia são diferentes.
Métrica	Volume garantido de cada sessão e do programa (Secção 3.4).

Tabela 5: Propriedades semânticas que a sintaxe não decide.

3.2 Gramática de atributos

Uma gramática de atributos acrescenta informação aos nós da árvore. **Herdados** descem do pai para os filhos (por exemplo, os nomes já declarados); **sintetizados** sobem dos filhos para o pai (por exemplo, o tipo de uma expressão).

Atributos usados:

- `env`: lista dos nomes declarados e respetivos tipos. Cada bloco `{ ... }` cria uma lista nova. `env_in` é o ambiente à entrada e `env_out` à saída.
- `type`: tipo de uma expressão. O tipo `error` marca expressões com erro e é aceite em qualquer lugar, para não repetir erros.
- `cv`: valor de uma expressão, se já for conhecido antes de executar. Só as constantes o têm.
- `valid`: a parte do programa está correta.
- `vol`: volume de treino (Secção 3.4).

Símbolos	Herdados	Sintetizados
start		valid, vol
decls, decl, var_decl, const_decl, exercise_decl, stmts, stmt	env_in	env_out, valid, vol
routine_decl	env_in	valid, vol
weeks	env_in	count, valid, vol
week	env_in, expected	valid, vol
days	env_in	names, valid, vol
day	env_in, seen	name, valid, vol
sessions	env_in	names, valid, vol
session	env_in, seen	name, valid, vol
block, assign, perform, rest, if_stmt, repeat, circuit	env_in	valid, vol
loadopt, rpeopt	env_in	cv, valid
expr, …, atom	env_in	type, cv, valid

Tabela 6: Atributos de cada símbolo.

`count`, `expected`, `names` e `seen` servem para contar semanas e detetar dias e sessões repetidos.

As regras usam funções auxiliares com nomes descritivos: `declared` e `declared_here` (o nome existe?), `type_of`, `kind_of` e `cv_of` (tipo, papel e valor de um nome), `compat`, `numeric`, `integer`, `boolean` e `comparable` (verificam tipos), `arith` (tipo de uma conta) e `fold` (calcula uma conta se os operandos são conhecidos).

Cada regra tem as equações em `ER { ... }`. Quando um símbolo aparece várias vezes, `[1]` é o da esquerda e `[2]`, `[3]`, … os da direita. As regras que só copiam atributos foram omitidas. A especificação completa de `valid` está em `liftplan-w-ag.lark`.

```
start : "program" ID "{" decls "}"
ER { decls.env_in = push(BUILTINS);       // começa com last_rpe e
fatigue
     start.valid  = decls.valid; }

decls : decls decl
ER {
    decls[2].env_in  = decls[1].env_in;
    decl.env_in      = decls[2].env_out;  // cada declaração vê as
anteriores
    decls[1].env_out = decl.env_out;
    decls[1].valid   = decls[2].valid and decl.valid;
}
| ER { decls.env_out = decls.env_in;  decls.valid = True; }

const_decl : "const" ID ":" type "=" expr ";"
ER {
    expr.env_in = const_decl.env_in;
```

```
    const_decl.valid = expr.valid
                       and not declared_here(const_decl.env_in,
str(ID))  // nome novo
                       and compat(type.t,
expr.type);                    // tipo certo
    const_decl.env_out = insert(const_decl.env_in, str(ID),
        (type.t, "const", expr.cv if compat(type.t, expr.type) else
None));
}

block : "{" stmts "}"
ER { stmts.env_in = push(block.env_in);   // cada bloco tem os seus
nomes
     block.valid  = stmts.valid; }

assign : ID "=" expr ";"
ER {
    expr.env_in  = assign.env_in;
    assign.valid = expr.valid
                   and declared(assign.env_in, str(ID))             //
existe
                   and kind_of(assign.env_in, str(ID)) == "var"     //
é variável
                   and compat(type_of(assign.env_in, str(ID)),
expr.type);
}

perform : "do" ID "sets" expr "reps" expr loadopt rpeopt ";"
ER {
    perform.valid = (kind_of(perform.env_in, str(ID)) ==
"exercise")  // é exercício
                    and expr[1].valid and expr[2].valid
                    and integer(expr[1].type) and
integer(expr[2].type)
                    and loadopt.valid and rpeopt.valid;
}

rpeopt : "rpe" expr
ER { rpeopt.cv = expr.cv;
     rpeopt.valid = expr.valid and numeric(expr.type)
                    and (expr.cv is None or (1 <= expr.cv and expr.cv
<= 10)); }
| ER { rpeopt.cv = None;  rpeopt.valid = True; }
```

Listagem 5: Atributos: declarações e instruções (excerto).

```
weeks : weeks week
ER {
    week.expected  = weeks[2].count + 1;   // a semana seguinte à
última
```

```
    weeks[1].count = weeks[2].count + 1;
    weeks[1].valid = weeks[2].valid and week.valid;
}
| week ER { week.expected = 1;  weeks.count = 1;  weeks.valid =
week.valid; }

week : "week" INT "{" days "}"
ER { days.env_in = push(week.env_in);
     week.valid  = (int(INT) == week.expected) and days.valid; }

days : days day
ER {
    day.seen      = days[2].names;                  // dias já usados
nesta semana
    days[1].names = days[2].names | set([day.name]);
    days[1].valid = days[2].valid and day.valid;
}
| day ER { day.seen = set();  days.names = set([day.name]);
days.valid = day.valid; }

day : DAY "{" sessions "}"
ER { sessions.env_in = day.env_in;
     day.name  = str(DAY);
     day.valid = (day.name not in day.seen) and sessions.valid; }  //
dia não repetido

mul_e : mul_e SLASH unary_e
ER {
    mul_e[1].type  = arith(mul_e[2].type, unary_e.type);
    mul_e[1].cv    = fold("/", mul_e[1].type, mul_e[2].cv,
unary_e.cv);
    mul_e[1].valid = mul_e[2].valid and unary_e.valid
                     and numeric(mul_e[2].type) and
numeric(unary_e.type)
                     and (unary_e.cv !=
0);                              // não divide por zero
}

atom : ID
ER {
    atom.type  = type_of(atom.env_in, str(ID));   // procura o nome no
ambiente
    atom.cv    = cv_of(atom.env_in, str(ID));
    atom.valid = declared(atom.env_in, str(ID))
                 and kind_of(atom.env_in, str(ID)) != "exercise";
}
```

Listagem 6: Atributos: semanas, dias, sessões e expressões (excerto).

3.3 Ordem de cálculo

Os atributos herdados só dependem do pai e dos irmãos à **esquerda** (a semana 2 só precisa de saber quantas semanas vieram antes). Uma gramática assim chama-se **L-atribuída**, e todos os atributos podem ser calculados numa só passagem pela árvore, da esquerda para a direita.

Exemplo, com `const a : int = 2; var b : float = a * 1.5;` num bloco:

1. O bloco cria um ambiente novo e passa-o a `const a`.
2. `const a` acrescenta `a` (tipo `int`, valor 2).
3. Em `var b`, a expressão `a * 1.5` tem tipo `float` e valor `3.0`.
4. `var b` acrescenta `b` **sem** valor conhecido, porque uma variável pode mudar.

3.4 Volume garantido

O `vol` calcula o volume de treino. O volume de um `do` é `sets * reps * load` (sem `load`, usa-se 1). Conta só o que se sabe antes de executar, por isso é um **mínimo garantido**: valores que dependem de variáveis ou de `last_rpe` não entram. As suas equações estão aqui e em `liftplan.py`, mas não em `liftplan-w-ag.lark`.

```
perform.vol  = expr[1].cv * expr[2].cv * loadopt.cv
               if None not in (expr[1].cv, expr[2].cv, loadopt.cv)
else 0
assign.vol = 0;  rest.vol = 0;  var_decl.vol = 0;  const_decl.vol = 0
if_stmt.vol  = 0                                  // if sem else
if_stmt.vol  = min(block[1].vol, block[2].vol)  // if com else: o
menor dos ramos
repeat.vol   = expr.cv * block.vol if integer(expr.type) and expr.cv
is not None else 0
stmts[1].vol = stmts[2].vol + stmt.vol          // soma das instruções
```

Listagem 7: Equações de `vol` (excerto).

Sessões, dias, semanas e o programa somam os volumes das suas partes.

3.5 Implementação

A gramática de atributos foi implementada em Python, com a biblioteca Lark (`liftplan.py`). O programa percorre a árvore uma vez, com uma função por regra:

- O ambiente é uma pilha: `push` ao entrar num bloco, `pop` ao sair.
- `type` e `cv` são o resultado da função de cada expressão, e `vol` o das instruções.
- Em vez de `valid`, guarda uma lista de erros com número de linha, o que mostra vários erros de uma vez.

4 Qualidade da Linguagem e da Gramática

4.1 Qualidade da linguagem

Propriedade	**Avaliação**
Adequação ao domínio	Boa. As palavras da linguagem (`exercise`, `session`, `sets`, `reps`, `load`, `rpe`, `rest`) vêm do treino de força. Limitação: cargas e descansos não têm unidade.
Legibilidade	Boa. Uma prescrição lê-se quase como uma frase (`do Squat sets 4 reps 5 load 100.0 rpe 7;`). Há chavetas, palavras-chave claras e comentários `//`.
Facilidade de escrita	Razoável. Constantes, `repeat` e `circuit` evitam repetição, mas não se pode reutilizar uma sessão noutra semana.
Simplicidade	Boa. Só há quatro tipos e 28 palavras-chave. Uma expressão pode ser usada em qualquer sítio que peça um valor.
Fiabilidade	Boa. A verificação deteta erros de nomes, tipos, valores e estrutura (Tabela 5) e indica a linha de cada um.
Expressividade	Moderada, de propósito. Todos os ciclos têm contagem fixa, por isso o programa termina sempre. Não há listas nem funções.

Tabela 7: Avaliação da qualidade da linguagem.

4.2 Qualidade da gramática

Propriedade	**Avaliação**
Sem ambiguidade	Cada programa tem uma só leitura possível. A precedência dos operadores está definida na gramática (Tabela 4) e as chavetas obrigatórias evitam dúvidas sobre a que `if` pertence um `else`. O Lark aceitou a gramática sem conflitos.
Análise simples	O programa é lido da esquerda para a direita, sem voltar atrás. Cada instrução começa por uma palavra-chave própria (ou por um nome seguido de `=`), por isso é sempre claro qual é.
Precedência	Correta: `1 + 2 * 3` é lido como `1 + (2 * 3)` e `a - b - c` como `(a - b) - c`.
Sem repetições	`var_decl` e `const_decl` são usadas em dois sítios, e `repeat` e `circuit` partilham a mesma forma.
Modularidade	Três partes separadas: estrutura, instruções e expressões. Acrescentar um operador só mexe nas expressões.
Sintaxe e semântica separadas	A gramática só define a forma. Declarações, tipos e repetições de dias ou sessões são verificados à parte (Secção 3), o que a mantém pequena.
Palavras-chave	Reservadas sem afetar nomes parecidos: `reps_base` é um nome válido.

Tabela 8: Avaliação da qualidade da gramática.

4.3 Limitações

Faltam unidades nas cargas e nos descansos, não se podem reutilizar sessões, as chaves dos atributos de `exercise` não são validadas e o volume é só um mínimo, porque ignora valores conhecidos apenas na execução.